

EduGenie: Google Gemini Powered Learning Assistant

2. Requirement Analysis

This document translates the supplied EduGenie description into functional, technical, interface, and operational requirements without adding requirements that are absent from the source.

2.1 Functional Requirements

The system shall accept educational questions and return AI-generated answers.

The system shall explain concepts in simplified and readable language.

The system shall generate quizzes from a topic or passage.

The system shall summarize long educational content while retaining core information.

The system shall generate personalized learning recommendations.

The quiz output shall contain three MCQs, four options for each question, and a correct answer.

2.2 Backend Requirements

FastAPI shall provide the backend API.

RESTful endpoints shall be defined for /qa, /explain, /quiz, /summarize, and /learn/recommendations.

Each endpoint shall connect to the corresponding module logic.

2.3 AI Requirements

Gemini 1.5 Pro via API shall support Q&A, summarization, quiz generation, and learning paths.

LaMini-Flan-T5-783M shall support concept explanation.

The quiz implementation shall clean Markdown code blocks before parsing JSON and shall provide an error message if generation or parsing fails.

2.4 Frontend Requirements

The frontend shall use HTML and CSS.

The interface shall provide a task dropdown containing Explain, QnA, Quiz, Summary, and Recommend Path.

The interface shall contain a text area, submit button, and result area.

The interface shall use responsive styling.

2.5 Environment and Prerequisites

Python 3.10+.

FastAPI framework.

HTML and CSS basics for the frontend.

Google Gemini API key.

Uvicorn ASGI server.

Jinja2 HTML templating engine.

2.6 Operational Requirements

The documented local execution command is: `uvicorn main:app --reload`.

The documented local URL is `http://127.0.0.1:8000`.

Functional testing shall cover asking a question, getting an explanation, generating a quiz, summarizing content, and obtaining a personalized learning plan.

Conclusion

The requirements provide a clear boundary for implementation: a lightweight web UI, FastAPI service layer, five AI-assisted functions, and the documented local development environment.

Source basis: EduGenie project document supplied with this request and the repository structure shown in the supplied reference image. Where the source did not specify an implementation detail, this documentation preserves the documented scope rather than inventing project facts.